

Trabajando con archivos

Santiago Gallego

2026-05-07

Contenido

0.1 Ejercicio 1: Limpiando un inventario forestal (I/O y Excepciones)

A menudo recibimos datos de campo tomados manualmente que contienen errores tipográficos. Crea un script que realice lo siguiente:

1. Crea programáticamente un archivo llamado `arboles_crudos.txt` con el siguiente contenido (nota los errores intencionales en las latitudes y longitudes):
2. Escribe una función llamada `limpiar_inventario` que abra este archivo, lea línea por línea (omitendo el encabezado) y utilice un bloque de manejo de excepciones (`try-except / tryCatch`) al intentar convertir las coordenadas a decimales (`float / numeric`).
3. Si la línea es válida, escríbela en un nuevo archivo llamado `arboles_limpios.csv`. Si falla, omítela pero imprime en consola una advertencia indicando qué especie tuvo el error.

0.1.1 python

```
# 1. Crear el archivo programáticamente
# Definimos el string con los datos, incluyendo errores tipográficos intencionales
contenido = """Especie,Latitud,Longitud
Roble,4.6097,-74.0817
Pino,4.6XYZ,-74.0900
Cedro,4.6150,-74.0850
Eucalipto,NULA,-74.0700
Nogal,4.6200,-74.0650"""

# 'w' abre el archivo en modo escritura (write)
with open("arboles_crudos.txt", "w") as f:
    f.write(contenido) # .write guarda el texto en el disco duro
```

135

```
def limpiar_inventario():
    try:
        # Abrimos el archivo de entrada para leer ('r') y el de salida para escribir ('w')
        with open("arboles_crudos.txt", "r") as entrada, open("arboles_limpios.csv", "w") as salida:
            lineas = entrada.readlines() # Lee todas las líneas y las guarda en una lista
            salida.write(lineas[0]) # Escribimos la primera línea (cabecera) directamente
```

```

    for linea in lineas[1:]: # Recorremos desde la segunda línea en adelante
        partes = linea.strip().split(",") # Limpia espacios y separa por comas
        especie = partes[0] # Extrae el nombre de la especie

        try:
            # Intento de conversión: si falla, saltará al bloque 'except'
            lat = float(partes[1]) # Convierte texto de latitud a decimal
            lon = float(partes[2]) # Convierte texto de longitud a decimal

            # Si no hubo error arriba, escribe la línea limpia en el nuevo archivo
            salida.write(f"{especie},{lat},{lon}\n")
        except ValueError:
            # Captura errores de conversión (como "4.6XYZ" o "NULA")
            print(f"Advertencia: Omitiendo registro de '{especie}' por coordenadas inválidas.")
        except FileNotFoundError:
            # Captura el error en caso de que el archivo 'arboles_crudos.txt' no exista
            print("Error: El archivo de entrada no existe.")

# Ejecución de la función
limpiar_inventario()

```

Advertencia: Omitiendo registro de 'Pino' por coordenadas inválidas.

Advertencia: Omitiendo registro de 'Eucalipto' por coordenadas inválidas.

0.1.2 R

```

# 1. Crear el archivo programáticamente
# Definimos la cadena de texto con saltos de línea
contenido <- "Especie,Latitud,Longitud
Roble,4.6097,-74.0817
Pino,4.6XYZ,-74.0900
Cedro,4.6150,-74.0850
Eucalipto,NULA,-74.0700
Nogal,4.6200,-74.0650"

writeLines(contenido, "arboles_crudos.txt") # Escribe el contenido en un archivo de texto

limpiar_inventario <- function() {
  # Leer líneas del archivo de texto
  lineas <- readLines("arboles_crudos.txt")
  encabezado <- lineas[1] # Extrae la primera línea
  registros <- lineas[-1] # Guarda todo excepto la primera línea

  # Crea el archivo de salida inicial con el encabezado
  writeLines(encabezado, "arboles_limpios.csv")
}

```

```

for (linea in registros) {
  partes <- strsplit(linea, ",")[[1]] # Divide la cadena por comas
  especie <- partes[1]                # Obtiene el nombre del árbol

  # Bloque de manejo de errores en R
  resultado <- tryCatch({
    lat <- as.numeric(partes[2]) # Intenta convertir a número
    lon <- as.numeric(partes[3]) # Intenta convertir a número

    # En R, as.numeric devuelve NA si falla. Forzamos un error manual si hay NA
    if (is.na(lat) || is.na(lon)) stop("Error de conversión")

    # Si los datos son válidos, los añade al archivo CSV (append = TRUE)
    cat(paste0(especie, ",", lat, ",", lon, "\n"), file = "arboles_limpios.csv", append = TRUE)
    TRUE # Indica éxito para el bloque tryCatch
  }, error = function(e) {
    # Función que se ejecuta si ocurre un 'stop' o error de conversión
    cat(paste("Advertencia: Omitiendo registro de '", especie, "' por coordenadas inválidas.\n"))
    FALSE # Indica fallo
  })
}

# Ejecución de la función
limpiar_inventario()

```

Warning in doTryCatch(return(expr), name, parentenv, handler): NAs introduced by coercion

Advertencia: Omitiendo registro de ' Pino ' por coordenadas inválidas.

Warning in doTryCatch(return(expr), name, parentenv, handler): NAs introduced by coercion

Advertencia: Omitiendo registro de ' Eucalipto ' por coordenadas inválidas.

0.2 Ejercicio 2: Analizando datos estructurados (Diccionarios y Listas)

Vamos a tomar los datos limpios y pasarlos a la memoria estructurada de nuestro programa:

1. Crea una función `cargar_datos_espaciales` que lea el archivo `arboles_limpios.csv` que generaste en el Ejercicio 1.
2. Por cada línea, crea un Diccionario (o Lista nombrada en R) con las llaves `especie`, `latitud` y `longitud`.

3. Almacena todos esos diccionarios en una Lista o Arreglo principal.
4. Recorre esa lista final y calcula programáticamente cuál es la latitud promedio de los árboles válidos, imprimiendo el resultado en consola.

0.2.1 python

```
def cargar_datos_espaciales():
    inventario = [] # Lista donde guardaremos los diccionarios de cada árbol

    try:
        with open("arboles_limpios.csv", "r") as f:
            lineas = f.readlines() # Leemos el archivo limpio generado anteriormente

            for linea in lineas[1:]: # Omitimos la cabecera
                datos = linea.strip().split(",") # Separamos los valores
                # Creamos una estructura de par clave-valor (diccionario)
                arbol = {
                    "especie": datos[0],
                    "latitud": float(datos[1]),
                    "longitud": float(datos[2])
                }
                inventario.append(arbol) # Guardamos el diccionario en la lista principal

            # Cálculo estadístico simple recorriendo la lista de diccionarios
            if inventario:
                # Sumamos todas las latitudes usando una expresión generadora
                suma_lats = sum(arbol["latitud"] for arbol in inventario)
                promedio = suma_lats / len(inventario) # Calculamos la media
                print(f"Se cargaron {len(inventario)} árboles válidos.")
                print(f"La latitud promedio es: {promedio:.4f}")

            return inventario # Retornamos la estructura de datos cargada

    except FileNotFoundError:
        print("Error: No se encontró el archivo 'arboles_limpios.csv'.")

# Ejecutamos y guardamos el resultado en una variable global
lista_arboles = cargar_datos_espaciales()
```

Se cargaron 3 árboles válidos.
La latitud promedio es: 4.6149

0.2.2 R

```

cargar_datos_espaciales <- function() {
  # Validación de existencia de archivo
  if (!file.exists("arboles_limpios.csv")) {
    stop("Error: El archivo 'arboles_limpios.csv' no existe.")
  }

  # Leemos el CSV directamente a una tabla (Data Frame)
  datos_tabla <- read.csv("arboles_limpios.csv", stringsAsFactors = FALSE)
  inventario <- list() # Lista principal (colección de objetos)

  # Iteramos por cada fila de la tabla cargada
  for (i in 1:nrow(datos_tabla)) {
    # Creamos una lista nombrada (equivalente a diccionario) para cada árbol
    arbol <- list(
      especie = datos_tabla$Especie[i],
      latitud = datos_tabla$Latitud[i],
      longitud = datos_tabla$Longitud[i]
    )
    inventario[[i]] <- arbol # Guardamos el objeto en la lista principal
  }

  # Cálculo de latitud promedio usando sapply para extraer todas las latitudes
  latitudes <- sapply(inventario, function(a) a$latitud)
  promedio <- mean(latitudes) # Función matemática para la media

  # Reporte de resultados en consola
  cat(sprintf("Se cargaron %d árboles válidos.\n", length(inventario)))
  cat(sprintf("La latitud promedio es: %.4f\n", promedio))

  return(inventario) # Retornamos la lista de listas
}

# Ejecutamos la función
lista_arboles <- cargar_datos_espaciales()

```

Se cargaron 3 árboles válidos.
 La latitud promedio es: 4.6149

0.3 Preguntas

0.3.1 Sobre el Ejercicio 1 (Manejo de Excepciones):

Explica por qué procesar datos espaciales usando bloques try...catch (o except) es una mejor práctica que dejar que el intérprete arroje un error por defecto. ¿Qué pasaría con un archivo de 500,000 registros si el registro número 499,999 tiene un error tipográfico y no usaste manejo de excepciones?

Procesar datos espaciales con manejo de excepciones es una mejor práctica por tres razones fundamentales:

1. Continuidad del Proceso: Los datos geográficos (GPS, sensores, encuestas de campo) son “sucios” por naturaleza. Un bloque try permite que el programa “salte” el obstáculo y siga procesando el resto de los datos. Sin esto, el programa es frágil.
2. Reporte de Errores (Logging): En lugar de que el programa muera con un mensaje técnico incomprensible, tú controlas la salida. Puedes imprimir: “Error en el árbol ID: 450, especie: Roble”. Esto facilita enormemente la corrección manual posterior.
3. Integridad de Archivos: Al capturar el error, puedes decidir qué escribir en el archivo de salida. Esto garantiza que el producto final (el archivo limpio) tenga un formato consistente y no contenga “basura” que rompa otros mapas o bases de datos.

El desastre de los 500,000 registros Si tienes medio millón de registros y el número 499,999 tiene un error (por ejemplo, una letra en una coordenada) y no usaste manejo de excepciones:

Pérdida de Tiempo: El intérprete arrojará un ValueError (en Python) o detendrá la ejecución (en R) justo antes de terminar. Habrás desperdiciado todo el tiempo de procesamiento de los 499,998 registros anteriores.

Corrupción de Salida: Si tu script estaba escribiendo en un archivo de salida línea por línea, el archivo quedará incompleto o “abierto”.

Frustración Operativa: En un flujo de trabajo automatizado, esto significa que el proceso fallará por completo a las 3:00 a. m., y nadie tendrá un mapa listo para la reunión de las 8:00 a. m. solo por un error tipográfico en la penúltima línea.

0.3.2 Sobre la Gestión de Recursos (I/O):

Compara cómo Python y Julia manejan el cierre automático de archivos (with open(...) as f: vs open(...) do f) frente al enfoque de R (close()). ¿Por qué es un riesgo de seguridad y rendimiento dejar conexiones de archivos abiertas en un flujo de geoprocesamiento continuo?

1. Comparativa de Control Python (with) y Julia (do): Utilizan gestores de contexto. Estos garantizan que el archivo se cierre automáticamente al finalizar el bloque, incluso si ocurre un error inesperado. Es una gestión “a prueba de fallos”.

R (close()): Depende de una gestión manual. El programador debe cerrar la conexión explícitamente. Si el script falla antes de ejecutar close(), el archivo permanece abierto en el sistema.

2. Riesgos de dejar archivos abiertos En flujos de geoprocesamiento continuo, no cerrar archivos genera tres problemas críticos:

Agotamiento de descriptores: El sistema operativo tiene un límite de archivos abiertos por proceso. Si superas ese límite, el programa se bloquea y no puede abrir más datos.

Pérdida de datos (Buffering): Los datos no se escriben físicamente en el disco en tiempo real, sino que se guardan en una memoria temporal (búfer). Si el archivo no se cierra correctamente, lo que está en el búfer se pierde y el archivo puede quedar corrupto.